ECON485 Fall 2025 – Homework Assignment 2

Course Registration System – SQL Homework

This homework continues from Homework Assignment 1. You will now work with SQL to create tables, insert data, and check rules. Please use your **Homework 1 Part 3a** design as the reference point for this homework.

Part 1 – Creating Tables, Inserting Data, and Showing Basic Actions

In Homework 1 Part 3a, you designed a 2NF database for a very simple course registration system.

This basic system had:

- Students
- Courses
- Sections
- Registrations (student to section link)

There were no rules such as prerequisites, limits, time conflicts, etc. In this homework, you will implement this simple system in SQL.

Task 1a – Create the Tables

Using your design from HW1 Part 3a: Write SQL CREATE TABLE statements for all tables. Include primary keys and foreign keys. Make sure field names and data types are reasonable and consistent.

If you changed your design after HW1, you may correct it here.

Task 1b – Insert Example Data

Insert the following minimum data:

- At least 3 courses
- Each course must have at least 1 section
- At least 10 students
- At least 15 total registrations after all actions are finished

Your system should show realistic data. (Example: real course codes, names, student names, etc.)

You must use:

- INSERT statements
- DELETE or UPDATE statements if needed

Task 1c – Demonstrate Add and Drop Actions

Using SQL statements, show how students add and drop courses:

- Each student must add at least one course section
- At least 3 students must drop a course section
- After all adds and drops, each course must have multiple students registered

Important:

- You do not need to implement business logic here.
- A simple INSERT into the registration table means “add.”
- A DELETE from the registration table means “drop.”

Task 1d – Show All Final Registrations

Write a SQL query to display the final state of course registrations:

- Show student name
- Show course code and section number
- Show any other helpful information
- You may use JOIN statements.

This is the end of Part 1.

Part 2 – Implementing Constraints from Homework 1 Part 4a

In Homework 1 Part 4a, you studied constraints that force a student to register for a course, such as:

- Prerequisites
- Co-requisites
- Required course sequences

In this homework, we will **only implement prerequisite constraints**.

Part 2a – SQL Constraints vs. Application Constraints

There are two ways to implement prerequisite rules.

1. **SQL-Enforced Constraints:** In this approach, the database server checks all rules. The database uses triggers, check constraints, and foreign keys. The system blocks incorrect registrations automatically. **Application code becomes simple. But the SQL rules can become very complex.** Also, every change to rules requires changing the database.
2. **Application-Enforced Constraints (with Assistive SQL):** This is the approach we will focus on. Here, the application software (Python, Java, etc.) checks the rules. **The SQL queries only help, by giving needed information.** The database does not block the action, but the software decides whether to allow it. The software runs SQL queries such as: “Did the student pass the prerequisite?”, “What grade did the student get in the previous course?”, “Is the course currently in progress?”. The application logic then chooses: ALLOW registration / BLOCK registration / SHOW WARNING message.

Why the second approach is easier for students in this course? Many students have limited experience with:

- full application development
- frameworks
- server-side validation
- business logic

Machine learning experience does not help much here either. This is because ML courses are usually “data in, data out” with no user interface and no business rules, as well as no permission or constraint logic.

So we use “assistive SQL”:
- SQL finds information
- A simple application layer makes the decision
- The SQL itself does not block anything

Because Homework 2 does not require writing Python/Java code, you will only write the SQL part of the logic.

Part 2b – Adding New Tables/Columns to Support Prerequisites

To check prerequisites, the database needs to store extra information.

Task 2a – Modify Your Design

Add the following (minimum):
- A table to store which course is the prerequisite of which course

Example fields:
- PrerequisiteID (PK)
- CourseID
- RequiredCourseID
- MinimumGrade

A field or table to store student grades. This can be a “CompletedCourses” table or a “Grade” column inside registrations (if you prefer).

Update your ER diagram to include:
- New tables
- New relationships
- **Updated foreign keys**

Make sure the diagram is clear and readable.

Part 2c – Implementing Assistive SQL (Prerequisite Check)

In this part, you write SQL statements that help the application decide whether a student is allowed to register. **You do not write any Python/Java code.** You only write SQL that would be used by an application.

Task 2b – Write Assistive SQL Queries

Write two SQL queries for the following:

1. Find All Prerequisites of a Course: Input as a course ID, and output as a list of all prerequisite courses, and required minimum grades.
2. Check Whether a Student Has Passed the Prerequisites: Input as a Student ID, and a Course ID; and output as the prerequisite course name, the student's grade for that course, whether the grade is acceptable. **If there are more than one prerequisites, there should be outputs for each prerequisite, not just the first one.**

With such SQL queries, the application can decide:

- “All prerequisites met → student may register”
- “Prerequisites not met → registration blocked”